



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table><tr><td>Student 1</td><td>Student 2</td></tr><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></table> Name 1: <u>JOANNE CHING YIN XUAN</u> Metric No: <u>A23CS0227</u> Link for Video Demo: https://youtu.be/Goz8dz6R5RI?feature=shared Name 2: <u>CHUA JIA LIN</u> Metric No: <u>A23CS0069</u> Link for Video Demo: https://youtu.be/hG5ElG-zxHo	Student 1	Student 2	<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>
Student 1	Student 2				
<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>				
SUBMISSION DATE & ITEMS:	Duration of submission :- 2 weeks Submission items in elearning:- 1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page. 2. The assembly programs (in .asm).				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax       ; LINE4
    mov var2, bx       ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

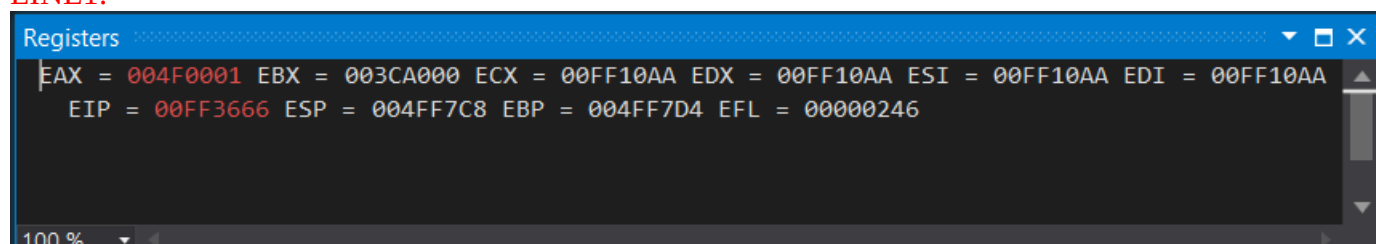
Answer Q1

- a) Fill (Write) in the contents for the related register in each line:

LINE 1	AX=0001h var1=000h	Move the value of var1(1d) into register AX
LINE 2	BX=0009h var2=0009h	Move the value of var2(9d) into register BX
LINE 3	AX=0009h BX=0001h	Exchange the value of register AX and BX
LINE 4	AX=0009h var1=0009h	Move the value of register AX into variable var1
LINE 5	BX=0001h var2=0001h	Move the value of register BX into variable var2

b) Paste here screenshot of all registers' content after each LINE is executed:

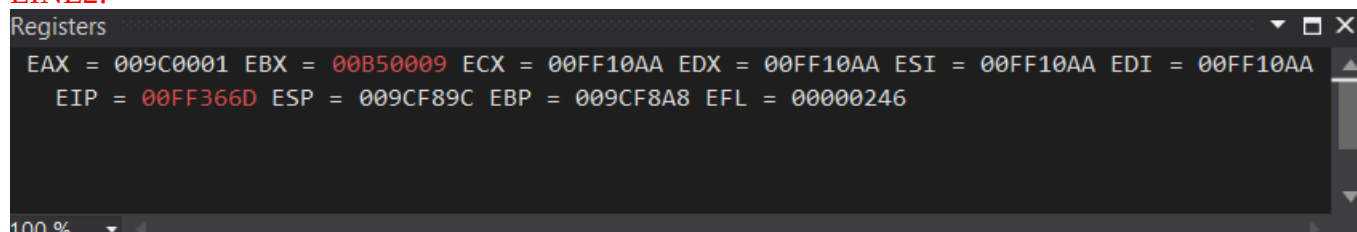
LINE1:



The screenshot shows the 'Registers' window with the following values: EAX = 004F0001, EBX = 003CA000, ECX = 00FF10AA, EDX = 00FF10AA, ESI = 00FF10AA, EDI = 00FF10AA, EIP = 00FF3666, ESP = 004FF7C8, EBP = 004FF7D4, and EFL = 00000246. The window has a blue title bar and a scrollbar on the right.

```
Registers
EAX = 004F0001 EBX = 003CA000 ECX = 00FF10AA EDX = 00FF10AA ESI = 00FF10AA EDI = 00FF10AA
EIP = 00FF3666 ESP = 004FF7C8 EBP = 004FF7D4 EFL = 00000246
```

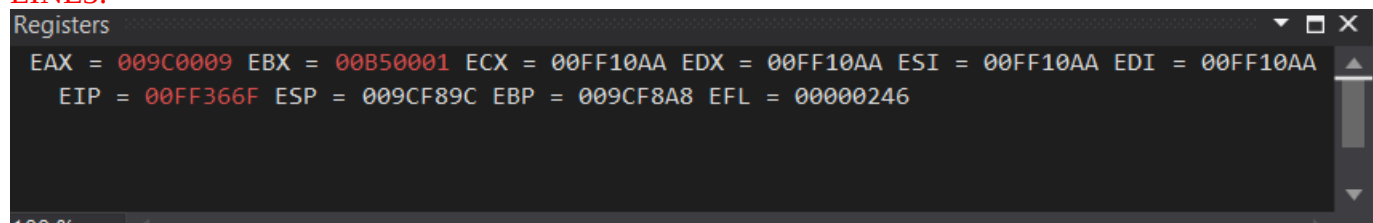
LINE2:



The screenshot shows the 'Registers' window with the following values: EAX = 009C0001, EBX = 00B50009, ECX = 00FF10AA, EDX = 00FF10AA, ESI = 00FF10AA, EDI = 00FF10AA, EIP = 00FF366D, ESP = 009CF89C, EBP = 009CF8A8, and EFL = 00000246. The window has a dark title bar and a scrollbar on the right.

```
Registers
EAX = 009C0001 EBX = 00B50009 ECX = 00FF10AA EDX = 00FF10AA ESI = 00FF10AA EDI = 00FF10AA
EIP = 00FF366D ESP = 009CF89C EBP = 009CF8A8 EFL = 00000246
```

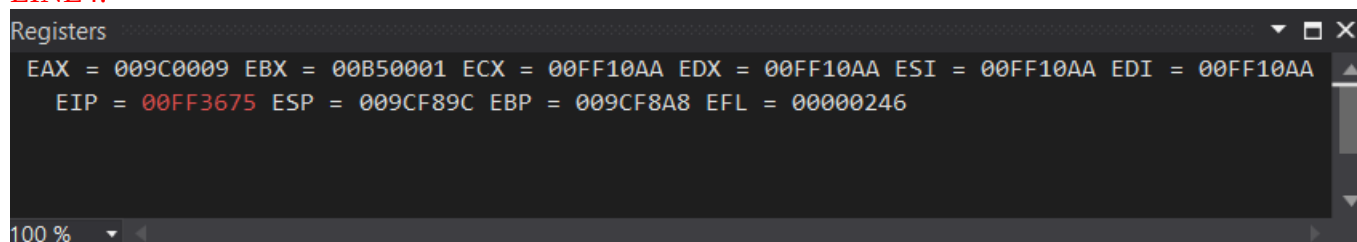
LINE3:



The screenshot shows the 'Registers' window with the following values: EAX = 009C0009, EBX = 00B50001, ECX = 00FF10AA, EDX = 00FF10AA, ESI = 00FF10AA, EDI = 00FF10AA, EIP = 00FF366F, ESP = 009CF89C, EBP = 009CF8A8, and EFL = 00000246. The window has a dark title bar and a scrollbar on the right.

```
Registers
EAX = 009C0009 EBX = 00B50001 ECX = 00FF10AA EDX = 00FF10AA ESI = 00FF10AA EDI = 00FF10AA
EIP = 00FF366F ESP = 009CF89C EBP = 009CF8A8 EFL = 00000246
```

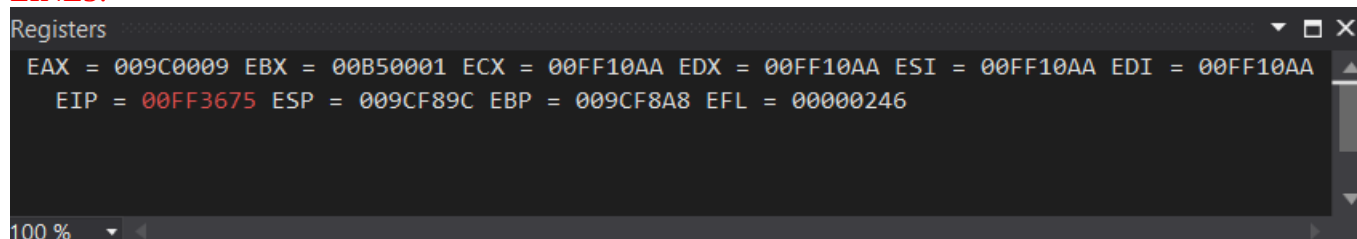
LINE4:



The screenshot shows the 'Registers' window with the following values: EAX = 009C0009, EBX = 00B50001, ECX = 00FF10AA, EDX = 00FF10AA, ESI = 00FF10AA, EDI = 00FF10AA, EIP = 00FF3675, ESP = 009CF89C, EBP = 009CF8A8, and EFL = 00000246. The window has a dark title bar and a scrollbar on the right.

```
Registers
EAX = 009C0009 EBX = 00B50001 ECX = 00FF10AA EDX = 00FF10AA ESI = 00FF10AA EDI = 00FF10AA
EIP = 00FF3675 ESP = 009CF89C EBP = 009CF8A8 EFL = 00000246
```

LINE5:



The screenshot shows the 'Registers' window with the following values: EAX = 009C0009, EBX = 00B50001, ECX = 00FF10AA, EDX = 00FF10AA, ESI = 00FF10AA, EDI = 00FF10AA, EIP = 00FF3675, ESP = 009CF89C, EBP = 009CF8A8, and EFL = 00000246. The window has a dark title bar and a scrollbar on the right.

```
Registers
EAX = 009C0009 EBX = 00B50001 ECX = 00FF10AA EDX = 00FF10AA ESI = 00FF10AA EDI = 00FF10AA
EIP = 00FF3675 ESP = 009CF89C EBP = 009CF8A8 EFL = 00000246
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

- a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx        ; LINE5
    inc eax            ; LINE6
    mov Rval,eax       ; LINE7
    exit
main endp
end main
```

Answer Q2

- a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX= 0000001Ah Xval= 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX= FFFFFFFE6h	Negate register EAX with two's complement
LINE3	EBX= 0000001Eh Yval= 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX= FFFFFFFF6h Zval= 00000028h	Subtract value of Zval (40d) from register EBX
LINE5	EAX=FFFFFFF0Ch EBX= FFFFFFFF6h	Subtract value of register EBX to register EAX
LINE6	EAX= FFFFFFFDDh	Increment of EAX by 1
LINE7	EAX= FFFFFFFDDh Rval= FFFFFFFDDh	Move value store in register EAX into variable Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 0000001A EBX = 00AA1000 ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 00961015 ESP = 00CFFCE8 EBP = 00CFFCF4 EFL = 00000246
```

LINE2:

```
Registers
EAX = FFFFFFFE EBX = 00AA1000 ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 00961017 ESP = 00CFFCE8 EBP = 00CFFCF4 EFL = 00000293

0x00964008 = 0000001E
```

LINE3:

```
Registers
EAX = FFFFFFFE EBX = 0000001E ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 0096101D ESP = 010FFAC0 EBP = 010FFACC EFL = 00000293

0x0096400C = 00000028
```

LINE4:

```
Registers
EAX = FFFFFFFE EBX = FFFFFFFF ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 00961023 ESP = 010FFAC0 EBP = 010FFACC EFL = 00000287
```

LINE5:

```
Registers
EAX = FFFFFFFD EBX = FFFFFFFF ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 00961025 ESP = 010FFAC0 EBP = 010FFACC EFL = 00000283
```

LINE6:

```
Registers
EAX = FFFFFFFD EBX = FFFFFFFF ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 00961026 ESP = 010FFAC0 EBP = 010FFACC EFL = 00000287

0x00964000 = 00000000
```

LINE7:

```
Registers
EAX = FFFFFFFD EBX = FFFFFFFF ECX = 00961005 EDX = 00961005 ESI = 00961005
EDI = 00961005 EIP = 00961028 ESP = 00F6FB30 EBP = 00F6FB3C EFL = 00000287
```

Q3. Execute the program below. Determine the output of the program by inspecting the content of the related registers.

- Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include Irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

- Fill (Write) in the contents for the related register in each line:

Table 3

LINE 1	EAX=00000005h var1=00000005h	Move the value of var1(5d) into register EAX
LINE 2	EAX=00000032h var2=0000000Ah	Multiply the value of var2(10d) into register EAX
LINE 3	EAX=00000046h var3=00000014h	Add the value of var3(20d) to register EAX
LINE 4	EAX=00000045h var4=00000045h	Decrement the value of register EAX by 1 and move value of register EAX into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 00000005 EBX = 00DF8000 ECX = 00271005 EDX = 00271005 ESI = 00271005 EDI = 00271005
EIP = 00271015 ESP = 00EFFB38 EBP = 00EFFB44 EFL = 00000246

0x00274004 = 0000000A
```

LINE2:

```
Registers
EAX = 00000032 EBX = 00DF8000 ECX = 00271005 EDX = 00000000 ESI = 00271005 EDI = 00271005
EIP = 0027101B ESP = 00EFFB38 EBP = 00EFFB44 EFL = 00000202

0x00274008 = 00000014
```

LINE3:

```
Registers
EAX = 00000046 EBX = 00DF8000 ECX = 00271005 EDX = 00000000 ESI = 00271005 EDI = 00271005
EIP = 00271021 ESP = 00EFFB38 EBP = 00EFFB44 EFL = 00000202
```

LINE4:

```
Registers
EAX = 00000045 EBX = 00DF8000 ECX = 00271005 EDX = 00000000 ESI = 00271005 EDI = 00271005
EIP = 00271022 ESP = 00EFFB38 EBP = 00EFFB44 EFL = 00000202
```

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

- Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX= 0028h var1= 0028h	Move the value of var1 (40d) into register AX
LINE2	BX= 0005h	Move the immediate value (5d) into register BX
LINE3	AX= 00C8h BX= 0005h	Multiply the value of register BX to register AX and store the value in register DX:AX
LINE4	BX= 000Ah var2= 000Ah	Move the value of var2 (10d) into register BX
LINE5	BX= 0007h	Subtract the value in register BX with an immediate value (3d)

LINE6	AX= 001Ch BX= 0007h DX= 0004h	Divide the register AX by register BX, the quotient is then store in register AX and the remainder store in register DX
LINE7	AX= 001Ch var4= 001Ch	Move the value of register AX into variable var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```

Registers
EAX = 006F0028 EBX = 00438000 ECX = 00C61005 EDX = 00C61005 ESI = 00C61005
EDI = 00C61005 EIP = 00C61016 ESP = 006FF7C4 EBP = 006FF7D0 EFL = 00000246
  
```

LINE2:

```

Registers
EAX = 006F0028 EBX = 00438005 ECX = 00C61005 EDX = 00C61005 ESI = 00C61005
EDI = 00C61005 EIP = 00C6101A ESP = 006FF7C4 EBP = 006FF7D0 EFL = 00000246
  
```

LINE3:

```

Registers
EAX = 006F00C8 EBX = 00438005 ECX = 00C61005 EDX = 00C60000 ESI = 00C61005
EDI = 00C61005 EIP = 00C6101D ESP = 006FF7C4 EBP = 006FF7D0 EFL = 00000202
  
```

LINE4:

```

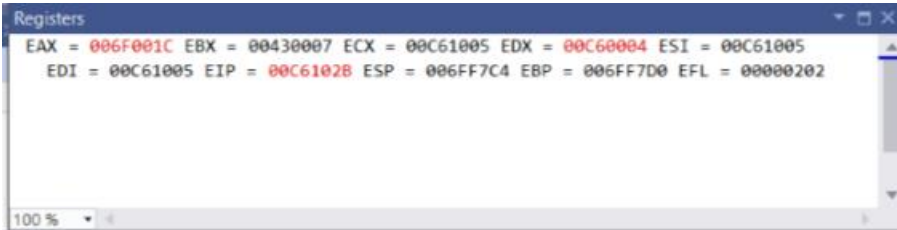
Registers
EAX = 006F00C8 EBX = 0043800A ECX = 00C61005 EDX = 00C60000 ESI = 00C61005
EDI = 00C61005 EIP = 00C61024 ESP = 006FF7C4 EBP = 006FF7D0 EFL = 00000202
  
```

LINE5:

```

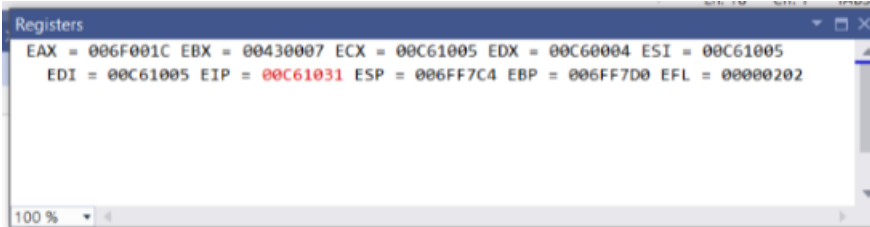
Registers
EAX = 006F00C8 EBX = 00438007 ECX = 00C61005 EDX = 00C60000 ESI = 00C61005
EDI = 00C61005 EIP = 00C61028 ESP = 006FF7C4 EBP = 006FF7D0 EFL = 00000202
  
```

LINE6:



A screenshot of the Windows Task Manager's 'Registers' tab. The window title is 'Registers'. It displays the following values: EAX = 006F001C, EBX = 00430007, ECX = 00C61005, EDX = 00C60004, ESI = 00C61005, EDI = 00C61005, EIP = 00C61028, ESP = 006FF7C4, EBP = 006FF7D0, and EFL = 00000202. The EIP register is highlighted in red. The window has a standard Windows XP-style title bar and a scrollbar on the right.

LINE7:



A screenshot of the Windows Task Manager's 'Registers' tab. The window title is 'Registers'. It displays the following values: EAX = 006F001C, EBX = 00430007, ECX = 00C61005, EDX = 00C60004, ESI = 00C61005, EDI = 00C61005, EIP = 00C61031, ESP = 006FF7C4, EBP = 006FF7D0, and EFL = 00000202. The EIP register is highlighted in red. The window has a standard Windows XP-style title bar and a scrollbar on the right.

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

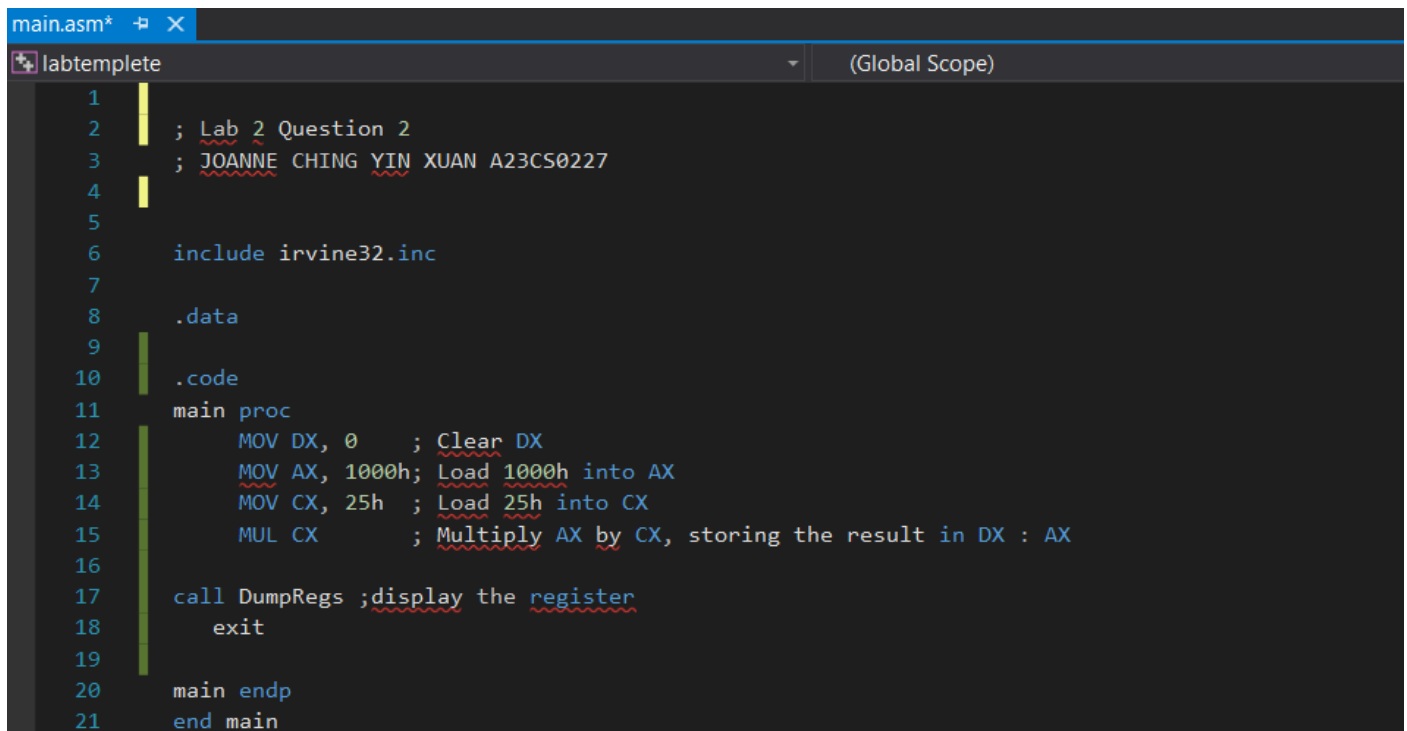
- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX,0      ; Clear DX
MOV AX,1000h   ; Load 1000h into AX
MOV CX, 25h    ; Load 25h into CX
MUL CX         ; Multiply AX by CX, storing the result in DX:AX
```

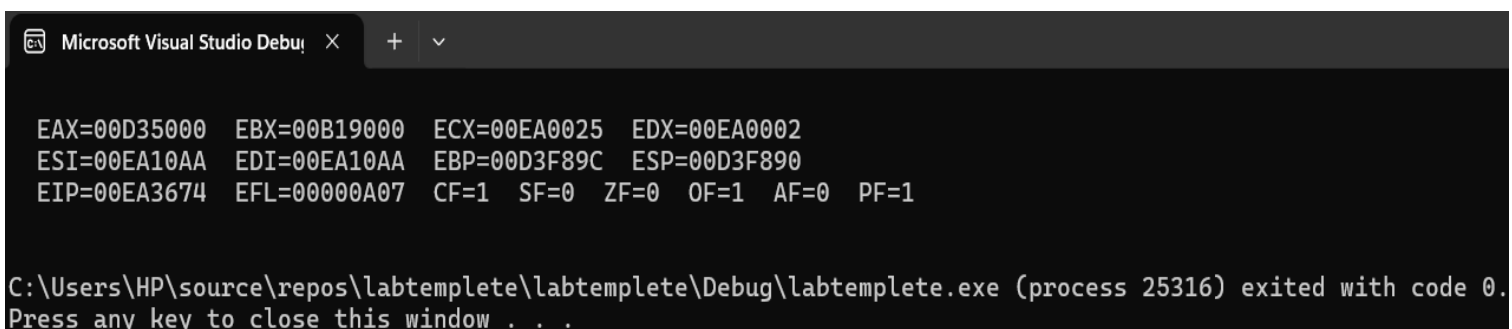
Answer Q5

- a) Screenshot of full program (.asm) :



```
main.asm*  X
labtemplate (Global Scope)
1
2 ; Lab 2 Question 2
3 ; JOANNE CHING YIN XUAN A23CS0227
4
5
6 include irvine32.inc
7
8 .data
9
10 .code
11 main proc
12     MOV DX, 0      ; Clear DX
13     MOV AX, 1000h   ; Load 1000h into AX
14     MOV CX, 25h    ; Load 25h into CX
15     MUL CX         ; Multiply AX by CX, storing the result in DX : AX
16
17     call DumpRegs ;display the register
18     exit
19
20 main endp
21 end main
```

- b) Paste here the screenshot of the final registers' content (DumpReg):



```
Microsoft Visual Studio Debug  X  +  v

EAX=00D35000  EBX=00B19000  ECX=00EA0025  EDX=00EA0002
ESI=00EA10AA  EDI=00EA10AA  EBP=00D3F89C  ESP=00D3F890
EIP=00EA3674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1

C:\Users\HP\source\repos\labtemplate\labtemplate\Debug\labtemplate.exe (process 25316) exited with code 0.
Press any key to close this window . . .
```

Q6. Given the following instructions as is Code Snippet 2.

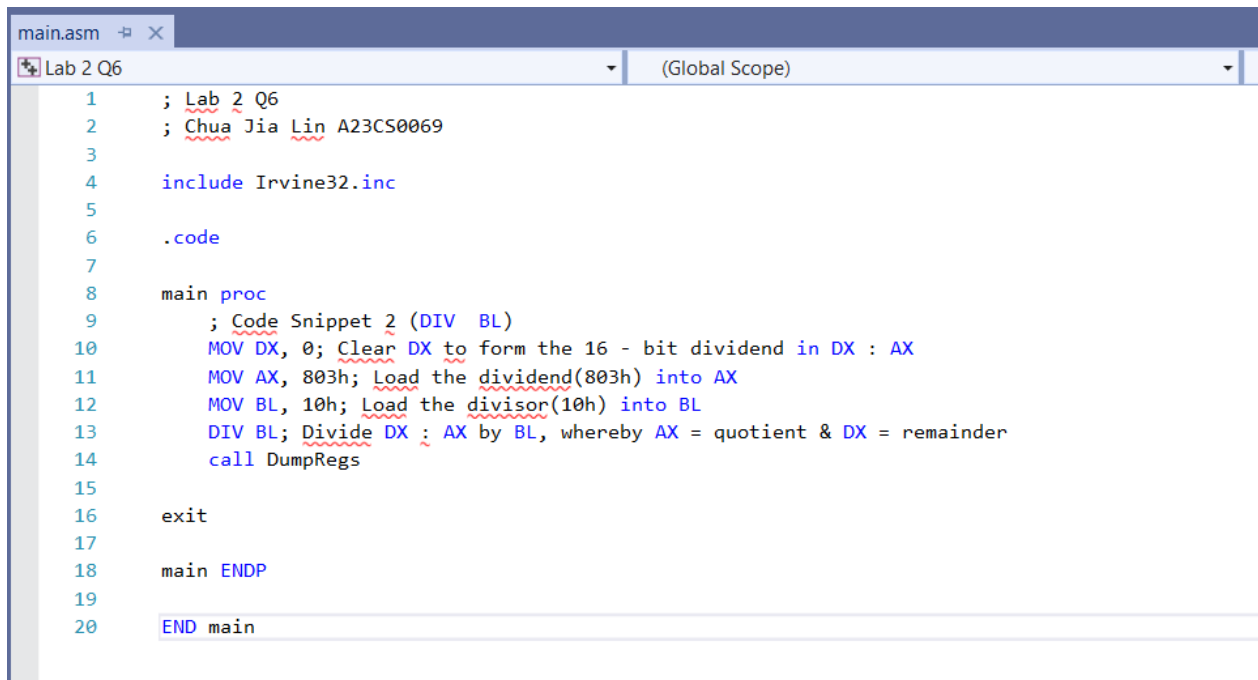
- Write a full program to execute the Code Snippet 2.
- What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

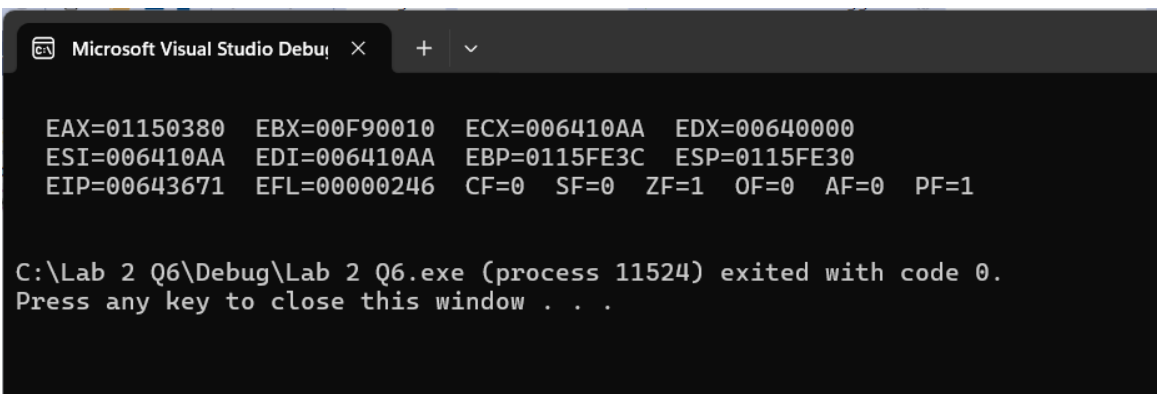
Answer Q6

- Screenshot of full program (.asm) :



```
main.asm x
Lab 2 Q6 (Global Scope)
1 ; Lab 2 Q6
2 ; Chua Jia Lin A23CS0069
3
4 include Irvine32.inc
5
6 .code
7
8 main proc
9 ; Code Snippet 2 (DIV BL)
10 MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
11 MOV AX, 803h; Load the dividend(803h) into AX
12 MOV BL, 10h; Load the divisor(10h) into BL
13 DIV BL; Divide DX : AX by BL, whereby AX = quotient & DX = remainder
14 call DumpRegs
15
16 exit
17
18 main ENDP
19
20 END main
```

- Paste here the screenshot of the final registers' content (DumpReg):



```
Microsoft Visual Studio Debug
EAX=01150380 EBX=00F90010 ECX=006410AA EDX=00640000
ESI=006410AA EDI=006410AA EBP=0115FE3C ESP=0115FE30
EIP=00643671 EFL=00000246 CF=0 SF=0 ZF=1 OF=0 AF=0 PF=1

C:\Lab 2 Q6\Debug\Lab 2 Q6.exe (process 11524) exited with code 0.
Press any key to close this window . . .
```